



Aluno: Gabriel Nemoto Gonçalves - CJ3037762

CÁLCULO RELACIONAL

Professor: Paulo Giovani de Faria Zeferino

IFSP - ADS CJOBDD1

Campos do Jordão – São Paulo

2026

Introdução

O cálculo relacional é um dos fundamentos teóricos dos bancos de dados relacionais, é uma linguagem de consulta declarativa de alto nível. Diferente de linguagens procedurais, ele especifica o que deve ser retornado, e não como obter o resultado, baseada na lógica de predicados de primeira ordem. Ao contrário da álgebra relacional, que foca nos passos da consulta (procedural), o cálculo relacional foca apenas em o que deve ser obtido, sem especificar o como. Esse conceito é essencial para compreender o funcionamento de linguagens como SQL e o modelo relacional proposto por Edgar F. Codd, que foi um cientista da computação britânico que ficou conhecido por criar o modelo relacional de bancos de dados no início da década de 1970. Trabalhando na IBM, ele propôs uma nova forma de organizar e manipular dados em tabelas, utilizando princípios matemáticos como a lógica e a teoria dos conjuntos. Suas ideias revolucionaram a área de bancos de dados e serviram de base para o desenvolvimento de sistemas modernos e da linguagem SQL, sendo fundamentais até os dias atuais.

O que é Cálculo Relacional

O Cálculo Relacional (CR) é uma linguagem de consulta formal. Utilizando-se de uma expressão declarativa pode-se especificar uma consulta. Uma expressão de cálculo permite a descrição da consulta desejada sem especificar os procedimentos para obtenção dessas informações, ou seja, é não procedural. Contudo, tal consulta deve ser capaz de descrever formalmente a informação desejada, com exatidão. Existem dois tipos: Cálculo Relacional de Tuplas (CRT) e Cálculo Relacional de Domínio (CRD). Eles são subconjuntos simples de lógica de primeira ordem. No Cálculo Relacional existem variáveis, constantes, operadores lógicos, de comparação e quantificadores. As expressões de Cálculo são chamadas de fórmulas. Uma tupla de respostas é essencialmente uma atribuição de constantes às variáveis que levam a fórmula a um estado verdadeiro. Em CRT, as variáveis são definidas sobre tuplas. Já em CRD, variáveis são definidas sobre o domínio dos elementos (ou seja, sobre os valores dos campos). Todas as expressões de consulta descritas em CR possuem equivalentes em Álgebra Relacional.

Onde o Cálculo Relacional é utilizado

O cálculo relacional é amplamente utilizado em diferentes áreas da computação, especialmente no contexto de bancos de dados e sistemas que manipulam grandes volumes de informação. Sua principal aplicação ocorre nos bancos de dados relacionais, onde sistemas como MySQL, PostgreSQL e Oracle utilizam conceitos derivados desse modelo. Nesses sistemas, os dados são organizados em tabelas que se relacionam entre si por meio de chaves, permitindo uma estrutura lógica e eficiente para armazenamento e recuperação de informações.

Além disso, o cálculo relacional está diretamente ligado às linguagens de consulta, como o SQL (Structured Query Language). Essa linguagem é baseada tanto no cálculo relacional quanto na álgebra relacional, permitindo que os usuários realizem consultas de forma declarativa, utilizando comandos como SELECT, WHERE e JOIN para especificar os dados desejados sem a necessidade de descrever o processo de obtenção.

O cálculo relacional também é amplamente empregado em sistemas de informação, incluindo sistemas empresariais como ERP (Enterprise Resource Planning) e CRM (Customer Relationship Management), além de aplicações web e sistemas bancários e financeiros. Esses sistemas dependem de bancos de dados estruturados para garantir o armazenamento seguro, consistente e eficiente das informações.

Por fim, sua aplicação se estende à área de ciência de dados e análise, onde é utilizado na consulta e manipulação de grandes volumes de dados estruturados. Nesse contexto, o cálculo relacional contribui para a extração de informações relevantes e para a tomada de decisões baseada em dados.

Porque seu estudo é importante

O estudo do cálculo relacional é fundamental por diversos motivos dentro da área de bancos de dados e sistemas de informação. Em primeiro lugar, ele constitui a base teórica dos bancos de dados relacionais, pois fundamenta o modelo relacional responsável por organizar os dados em tabelas inter-relacionadas, garantindo uma estrutura lógica e consistente.

Além disso, o conhecimento em cálculo relacional contribui para uma melhor compreensão da linguagem SQL, permitindo entender como as consultas são interpretadas e otimizadas pelos sistemas gerenciadores de banco de dados. Isso é essencial para desenvolver consultas mais eficientes e adequadas às necessidades do sistema.

Outro ponto importante é a abstração proporcionada por esse modelo. O cálculo relacional permite que o usuário descreva apenas o que deseja obter como resultado, sem precisar se preocupar com os detalhes de implementação. Isso facilita o desenvolvimento de consultas e contribui para a otimização do desempenho.

Por fim, o cálculo relacional também está diretamente relacionado à consistência e integridade dos dados. O modelo relacional estabelece regras que ajudam a manter os dados corretos, organizados e confiáveis, o que é essencial para qualquer sistema que dependa de informações precisas.

Exemplos Cálculo Relacional de Tuplas (CRT)

de como, ou em que ordem, avaliar uma consulta. Uma expressão de cálculo especifica o que deve ser recuperado em vez de como recuperá-lo. Portanto, o cálculo relacional é considerado uma linguagem não procedimental. Isso difere da álgebra relacional, na qual precisamos escrever uma sequência de operações para especificar uma solicitação de recuperação em uma ordem particular da aplicação das operações. Assim, ela pode ser considerada um modo procedimental de indicar uma consulta. É possível aninhar operações da álgebra para formar uma única expressão, porém, certa ordem entre as operações é sempre explicitamente especificada em uma expressão da álgebra relacional. Essa ordem também influencia a estratégia para avaliar a consulta. Uma expressão de cálculo pode ser escrita de maneiras diferentes, mas o modo como ela é escrita não tem relação com o modo como uma consulta deve ser avaliada.

Foi mostrado que qualquer recuperação que possa ser especificada na álgebra relacional básica também pode ser especificada no cálculo relacional e vice-versa. Em outras palavras, o poder expressivo das linguagens é idêntico. Isso levou à definição do conceito de uma linguagem *relacionalmente completa*. Uma linguagem de consulta relacional L é considerada *relacionalmente completa* se pudermos expressar em L qualquer consulta que possa ser expressa no cálculo relacional. A integralidade relacional se tornou uma base importante para comparar o poder expressivo das linguagens de consulta de alto nível. No entanto, como vimos na Seção 6.4, certas consultas frequentemente exigidas nas aplicações de banco de dados não podem ser expressas na álgebra ou no cálculo relacional básico. A maioria das linguagens de consulta relacional é *relacionalmente completa*, mas possui *mais poder expressivo* do que a álgebra relacional ou o cálculo relacional, por causa de operações adicionais como funções de agregação, agrupamento e ordenação. Conforme mencionamos na introdução deste capítulo, o cálculo relacional é importante por dois motivos. Primeiro, ele tem uma base firme na lógica matemática. Segundo, a linguagem de consulta padrão (SQL) para SGBDRs tem alguns de seus alicerces no cálculo relacional de tupla.

Nossos exemplos se referem ao banco de dados mostrado nas figuras 3.6 e 3.7. Usaremos as mesmas consultas da Seção 6.5. As seções 6.6.6, 6.6.7 e 6.6.8 discutem o tratamento com quantificadores universais e as questões de segurança de expressão. (Os alunos interessados em uma introdução básica ao cálculo relacional de tupla podem pular essas seções.)

6.6 O cálculo relacional de tupla

Nesta e na próxima seção, apresentamos outra linguagem de consulta formal para o modelo relacional, chamada *cálculo relacional*. Esta seção apresenta a linguagem conhecida como *cálculo relacional de tupla*, e a Seção 6.7 apresenta uma variação chamada *cálculo relacional de domínio*. Nas duas variações do cálculo relacional, escrevemos uma expressão declarativa para especificar uma solicitação de recuperação; logo, não existe descrição

rá uma sequência de operações em particular que corresponde a uma

6.6.1 Variáveis de tupla e relações de intervalo

O cálculo relacional de tupla é baseado na especificação de uma série de *variáveis de tupla*. Cada variável de tupla costuma *percorrer* determinada relação do banco de dados, significando que pode tomar como seu valor qualquer tupla individual dessa relação. Um cálculo relacional de tupla simples tem a forma:

$$\{t \mid \text{COND}(t)\}$$

onde t é uma variável de tupla e $\text{COND}(t)$ é uma expressão condicional (booleana) que envolve t e que é avaliada como TRUE ou FALSE para diferentes atribuições de tuplas à variável t . O resultado dessa consulta é o conjunto de todas as tuplas t que avaliam $\text{COND}(t)$ como TRUE. Diz-se que essas tuplas *satisfazem* $\text{COND}(t)$. Por exemplo, para encontrar todos os funcionários cujo salário é maior que R\$50.000,00, podemos escrever a seguinte expressão de cálculo de tupla:

$$\{t \mid \text{FUNCIONARIO}(t) \text{ AND } t.\text{Salario} > 50.000\}$$

A condição $\text{FUNCIONARIO}(t)$ especifica que a relação de intervalo da variável de tupla t é FUNCIONARIO . Cada tupla de FUNCIONARIO t que satisfaz a condição $t.\text{Salario} > 50.000$ será recuperada. Observe que $t.\text{Salario}$ referencia o atributo Salario da variável de tupla t . Tal notação é semelhante ao modo como os nomes de atributo são qualificados com nomes de relação ou apelidos em SQL, como vimos no Capítulo 4. Na notação do Capítulo 3, $t.\text{Salario}$ é o mesmo que escrever $t[\text{Salario}]$.

A consulta anterior recupera todos os valores de atributo para cada tupla t de FUNCIONARIO selecionada. Para recuperar apenas *alguns* dos atributos — digamos, o nome e o sobrenome —, escrevemos:

$$\{t.\text{Pnome}, t.\text{Unome} \mid \text{FUNCIONARIO}(t) \text{ AND } t.\text{Salario} > 50.000\}$$

Informalmente, precisamos especificar a seguinte informação em uma expressão de cálculo relacional de tupla:

- Para cada variável de tupla t , a relação de intervalo R de t . Esse valor é especificado por uma condição na forma $R(t)$. Se não especificarmos uma relação de intervalo, então a variável t oscilará por todas as tuplas possíveis 'no universo', pois ela não é restrita a nenhuma relação isolada.
- Uma condição para selecionar combinações de tuplas em particular. À medida que as va-

riáveis de tupla oscilam em suas respectivas relações de intervalo, a condição é avaliada em cada combinação possível de tuplas, a fim de identificar as combinações selecionadas para as quais a condição é avaliada como TRUE.

- Um conjunto de atributos a serem recuperados, os atributos solicitados. Os valores desses atributos são recuperados para cada combinação de tuplas selecionada.

Antes de discutirmos a sintaxe formal do cálculo relacional de tupla, considere outra consulta.

Consulta 0. Recuperar a data de nascimento e o endereço do funcionário (ou funcionários) cujo nome é João B. Silva.

C0: $\{t.Datanasc, t.Endereco \mid \text{FUNCIONARIO}(t) \text{ AND } t.Pnome = \text{'João'} \text{ AND } t.Minicial = \text{'B'} \text{ AND } t.Unome = \text{'Silva'}\}$

No cálculo relacional de tupla, primeiro especificamos os atributos solicitados $t.Datanasc$ e $t.Endereco$ para cada tupla t selecionada. Depois, especificamos a condição para selecionar uma tupla após a barra (|) — a saber, que t seja uma tupla da relação **FUNCIONARIO** cujos valores de atributo $Pnome$, $Minicial$ e $Unome$ são 'João', 'B' e 'Silva', respectivamente.

6.6.2 Expressões e fórmulas no cálculo relacional de tupla

Uma expressão geral do cálculo relacional de tupla tem a forma

$\{t_1, A_1, t_2, A_2, \dots, t_n, A_n \mid \text{COND}(t_1, t_2, \dots, t_n, t_{n+1}, t_{n+2}, \dots, t_{n+m})\}$

onde $t_1, t_2, \dots, t_n, t_{n+1}, \dots, t_{n+m}$ são variáveis de tupla, cada A_i é um atributo da relação em que t_i varia e **COND** é uma condição ou fórmula¹⁾ do cálculo relacional de tupla. Uma fórmula é composta de átomos de cálculo de predicado, que podem ser um dos seguintes:

- Um átomo da forma $R(t_i)$, onde R é um nome de relação e t_i é uma variável de tupla. Esse átomo identifica o intervalo da variável de tupla t_i como a relação cujo nome é R . Ele é avaliado como TRUE se t_i for uma tupla na relação R , e como FALSE em caso contrário.
- Um átomo da forma $t_i \text{ op } t_j, B$, onde op é um dos operadores de comparação no conjunto $\{=, <, <=, >, \geq, \neq\}$, t_i e t_j são variáveis de tupla,

A é um atributo da relação em que t_i varia, e B é um atributo da relação em que t_j varia.

- Um átomo da forma $t_i \text{ op } c$ ou $c \text{ op } t_j, B$, onde op é um dos operadores de comparação no conjunto $\{=, <, <=, >, \geq, \neq\}$, t_i e t_j são variáveis de tupla, A é um atributo da relação sobre a qual t_i varia, B é um atributo da relação em que t_j varia e c é um valor constante.

Cada um dos átomos anteriores é avaliado como TRUE ou FALSE para uma combinação específica de tuplas, o que é chamado de valor verdade do átomo. Em geral, uma variável de tupla t oscila em todas as tuplas possíveis no universo. Para átomos da forma $R(t_i)$, se t for atribuído a uma tupla que é um membro da relação especificada R , o átomo é TRUE; caso contrário, ele é FALSE. Em átomos dos tipos 2 e 3, se as variáveis de tupla forem atribuídas a tuplas de modo que os valores dos atributos especificados delas satisfaçam a condição, então o átomo é TRUE.

Uma fórmula (condição booleana) é composta de um ou mais átomos conectados por meio dos operadores lógicos AND, OR e NOT e é definida recursivamente pelas regras 1 e 2 da seguinte forma:

- Regra 1:** todo átomo é uma fórmula.

- Regra 2:** se F_1 e F_2 são fórmulas, então o mesmo ocorre para $(F_1 \text{ AND } F_2)$, $(F_1 \text{ OR } F_2)$, $\text{NOT}(F_1)$ e $\text{NOT}(F_2)$. Os valores verdade dessas fórmulas são derivados de suas fórmulas componentes F_1 e F_2 , como se segue:

- $(F_1 \text{ AND } F_2)$ é TRUE se tanto F_1 quanto F_2 forem TRUE; caso contrário, é FALSE.
- $(F_1 \text{ OR } F_2)$ é FALSE se tanto F_1 quanto F_2 forem FALSE; caso contrário, é TRUE.
- $\text{NOT}(F_1)$ é TRUE se F_1 for FALSE; é FALSE se F_1 for TRUE.
- $\text{NOT}(F_2)$ é TRUE se F_2 for FALSE; é FALSE se F_2 for TRUE.

6.6.3 Os quantificadores existenciais e universais

Além disso, dois símbolos especiais, chamados quantificadores, podem aparecer nas fórmulas: o quantificador universal ($\forall$) e o quantificador existencial ($\exists$). Os valores verdade para fórmulas com quantificadores são descritos nas regras 3 e 4 a seguir. Primeiro, definimos os conceitos de variáveis de tupla livre e ligada em uma fórmula. Informalmente, uma variável de tupla t é ligada se for quantificada, significando que ela aparece em uma cláusula

$(\exists t)$ ou $(\forall t)$; caso contrário, ela é livre. De maneira formal, definimos uma variável de tupla em uma fórmula como livre ou ligada de acordo com as seguintes regras:

- Uma ocorrência de uma variável de tupla em uma fórmula F que é um átomo é livre em F .
- Uma ocorrência de uma variável de tupla t é livre ou vinculada em uma fórmula composta de conectivos lógicos $\neg$ ($F_1 \text{ AND } F_2$), $(F_1 \text{ OR } F_2)$, $\text{NOT}(F_1)$ e $\text{NOT}(F_2)$ — dependendo dela ser livre ou ligada em F_1 ou F_2 (se ocorrer em uma delas). Observe que, em uma fórmula com a forma $F = (F_1 \text{ AND } F_2)$ ou $F = (F_1 \text{ OR } F_2)$, uma variável de tupla pode ser livre em F_1 e ligada em F_2 , ou vice-versa. Nesse caso, uma ocorrência da variável de tupla é ligada e a outra é livre em F .
- Todas as ocorrências livres de uma variável de tupla t em F são ligadas em uma fórmula F' da forma $F' = (\exists t)(F)$ ou $F' = (\forall t)(F)$. A variável de tupla é ligada ao quantificador especificado em F' . Por exemplo, considere as seguintes fórmulas:

$F_1: d.Dnome = \text{'Pesquisa'}$
 $F_2: (\exists t)(d.Dnumero = t.Dnr)$
 $F_3: (\forall d)(d.Cpl_gerente = \text{'333444555587'})$

A variável de tupla d é livre em F_1 e F_2 , embora seja ligada ao quantificador ($\forall$) em F_3 . A variável t é ligada ao quantificador ($\exists$) em F_2 .

Agora, podemos dar as regras 3 e 4 para a definição de uma fórmula que iniciamos anteriormente:

- Regra 3:** se F é uma fórmula, então o mesmo vale para $(\exists t)(F)$, onde t é uma variável de tupla. A fórmula $(\exists t)(F)$ é TRUE se a fórmula F é avaliada como TRUE para alguma (pelo menos uma) tupla atribuída a ocorrências livres de t em F ; caso contrário, $(\exists t)(F)$ é FALSE.
- Regra 4:** se F é uma fórmula, então o mesmo vale para $(\forall t)(F)$, onde t é uma variável de tupla. A fórmula $(\forall t)(F)$ é TRUE se a fórmula F é avaliada como TRUE para cada tupla (no universo) atribuída a ocorrências livres de t em F ; caso contrário, $(\forall t)(F)$ é FALSE.

O quantificador ($\exists$) é chamado de quantificador existencial porque uma fórmula $(\exists t)(F)$ é TRUE se existir alguma tupla que torne F TRUE. Para o quantificador universal, $(\forall t)(F)$ é TRUE se cada tupla possível que pode ser atribuída a ocorrências livres de t em F for substituída por t , e F é TRUE para cada substituição desse tipo. Ele é chamado quantificador

universal ou para cada porque cada tupla no universo de tuplas precisa tornar F TRUE para tornar a fórmula quantificada TRUE.

6.6.4 Exemplo de consultas no cálculo relacional de tupla

Usaremos algumas das mesmas consultas da Seção 6.5 para dar uma ideia de como elas são especificadas na álgebra relacional e no cálculo relacional. Observe que algumas consultas são mais fáceis de especificar na álgebra relacional do que no cálculo relacional, e vice-versa.

Consulta 1. Listar o nome e o endereço de todos os funcionários que trabalham para o departamento 'Pesquisa'.

C1: $\{t.Pnome, t.Unome, t.Endereco \mid \text{FUNCIONARIO}(t) \text{ AND } (\exists d)(\text{DEPARTAMENTO}(d) \text{ AND } d.Dnome = \text{'Pesquisa'} \text{ AND } d.Dnumero = t.Dnr))\}$

As únicas variáveis de tupla livres em uma expressão de cálculo relacional de tupla devem ser aquelas que aparecem à esquerda da barra (|). Em **C1**, t é a única variável livre; depois, ela é ligada sucessivamente a cada tupla. Se uma tupla satisfaz as condições especificadas após a barra em **C1**, os atributos $Pnome$, $Unome$ e $Endereco$ são recuperados para cada tupla desse tipo. As condições **FUNCIONARIO**(t) e **DEPARTAMENTO**(d) especificam as relações de intervalo para t e d . A condição $d.Dnome = \text{'Pesquisa'}$ é uma condição de seleção e corresponde a uma operação SELEÇÃO na álgebra relacional, ao passo que a condição $d.Dnumero = t.Dnr$ é uma condição de junção e semelhante em finalidade à operação JUNÇÃO INTERNA (ver Seção 6.3).

Consulta 2. Para cada projeto localizado em 'Mauá', listar o número do projeto, o número do departamento de controle e o sobrenome, data de nascimento e endereço do gerente do departamento.

C2: $\{p.Projnumero, p.Dnum, g.Unome, g.Datanasc, g.Endereco \mid \text{PROJETO}(p) \text{ AND } \text{FUNCIONARIO}(g) \text{ AND } p.Projlocalizacao = \text{'Mauá'} \text{ AND } (\exists d)(\text{DEPARTAMENTO}(d) \text{ AND } p.Dnum = d.Dnumero \text{ AND } d.Cpl_gerente = g.Cpl))\}$

Em **C2**, existem duas variáveis de tupla livres, p e g . A variável de tupla d é ligada ao quantificador existencial. A condição de consulta é avaliada para cada combinação de tuplas atribuídas a p e g , e de todas as combinações possíveis de tuplas às quais p e g estão ligadas, somente as combinações que satisfazem a condição são selecionadas.

¹⁾ Também chamada de fórmula bem formada, ou WFF (well-formed formula) em lógica matemática.

Diversas variáveis de tupla em uma consulta podem oscilar na mesma relação. Por exemplo, para especificar C8 — para cada funcionário, recupere o nome e último nome do funcionário e o nome e sobrenome de seu supervisor imediato —, determinamos duas variáveis de tupla f e s que percorrem a relação FUNCIONARIO:

C8: $\{f.Pnome, f.Unome, s.Pnome, s.Unome \mid$
 FUNCIONARIO(f) AND FUNCIONARIO(s)
 AND $f.Cpf_supervisor=s.Cpf\}$

Consulta 3'. Listar o nome do funcionário que trabalha em *algum* projeto controlado pelo departamento número 5. Essa é uma variação de C3 em que *todos* é mudado para *algum*. Neste caso, precisamos de duas condições de junção e dois quantificadores existenciais.

C0': $\{f.Unome, f.Pnome \mid$ FUNCIONARIO(f) AND
 $(\exists p)(\exists t)(PROJETO(p) \text{ AND TRABALHA_}$
 $EM(t) \text{ AND } p.Dnum=5 \text{ AND } t.Fcpl=f.Cpf$
 AND $p.Projnumero=t.Pnr)\}$

Consulta 4. Fazer uma lista dos números de projeto que envolvem um funcionário cujo último nome é 'Silva', seja como um trabalhador ou como um gerente do departamento de controle para o projeto.

C4: $\{p.Projnumero \mid PROJETO(p) \text{ AND } ((\exists f)(\exists t)$
 $(FUNCIONARIO(f) \text{ AND TRABALHA_}$
 $EM(t) \text{ AND } t.Pnr=p.Projnumero \text{ AND }$
 $f.Unome='Silva' \text{ AND } t.Fcpl=f.Cpf)\}$

OR

$((\exists g)(\exists d)(FUNCIONARIO(g) \text{ AND}$
 DEPARTAMENTO(d) AND $p.Dnum=d$,
 $Dnumero \text{ AND } d.Cpf_gerente=g.Cpf \text{ AND }$
 $g.Unome='Silva'))\}$

Compare isso com a versão da álgebra relacional dessa consulta na Seção 6.5. A operação UNIÃO na álgebra relacional normalmente pode ser substituída por um conectivo OR no cálculo relacional.

6.6.5 Notação para grafos de consulta

Nesta seção, descrevemos uma notação que foi proposta para representar as consultas do cálculo relacional que não envolvem quantificação complexa em uma forma gráfica. Esses tipos de consultas são conhecidos como *consultas seleção-projeção-junção*, pois envolvem essas três operações da álgebra relacional. A notação pode ser expandida para consultas mais gerais, mas não vamos discutir essas extensões aqui. Essa representação gráfica de uma consulta é chamada de *grafo de consulta*. A Figura 6.13 mostra o grafo de consulta para C2. As relações na consulta são representadas por nós de relação, que aparecem como círculos isolados. Valores constantes, normalmente das condições de seleção de consulta, são representados por nós de constante, que aparecem como círculos ou ovais duplos. As condições de seleção e junção são representadas pelas arestas do grafo (as linhas que conectam os nós), como mostra a Figura 6.13. Finalmente, os atributos a serem recuperados de cada relação são exibidos entre colchetes acima de cada uma delas.

A representação do grafo de consulta não indica uma ordem em particular para especificar quais operações realizar primeiro e, portanto, é uma representação mais neutra de uma consulta seleção-projeção-junção do que a representação na árvore de consulta (ver Seção 6.3.5), onde a ordem de execução é especificada de maneira implícita. Existe apenas um grafo de consulta correspondente a cada consulta. Embora algumas técnicas de otimização de consulta fossem baseadas em grafos de consulta, agora as árvores de consulta são preferíveis porque, na prática, o otimizador de consulta precisa mostrar a ordem das operações para a execução da consulta, o que não é possível nos grafos de consulta.

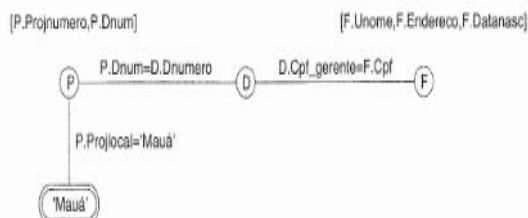


Figura 6.13
 Grafo de consulta para C2.

Na próxima seção, discutiremos o relacionamento entre os quantificadores universais e existenciais e mostraremos como um pode ser transformado no outro.

6.6.6 Transformando os quantificadores universais e existenciais

Agora, vamos apresentar algumas transformações bem conhecidas da lógica matemática que se relacionam aos quantificadores universais e existenciais. É possível transformar um quantificador universal em um quantificador existencial, e vice-versa, para obter uma expressão equivalente. Uma transformação geral pode ser descrita informalmente da seguinte forma: transformar um tipo de quantificador no outro com a negação (precedida por NOT); AND e OR substituem um ao outro; uma fórmula negada torna-se não negada; e uma fórmula não negada torna-se negada. Alguns casos especiais dessa transformação podem ser declarados da seguinte forma, onde o símbolo $\equiv$ significa *equivalente a*:

$$(\forall x)(P(x)) \equiv \text{NOT } (\exists x)(\text{NOT } (P(x)))$$

$$(\exists x)(P(x)) \equiv \text{NOT } (\forall x)(\text{NOT } (P(x)))$$

$$(\forall x)(P(x) \text{ AND } Q(x)) \equiv \text{NOT } (\exists x)(\text{NOT } (P(x)) \text{ OR } \text{NOT } (Q(x)))$$

$$(\forall x)(P(x) \text{ OR } Q(x)) \equiv \text{NOT } (\exists x)(\text{NOT } (P(x)) \text{ AND } \text{NOT } (Q(x)))$$

$$(\exists x)(P(x) \text{ OR } Q(x)) \equiv \text{NOT } (\forall x)(\text{NOT } (P(x)) \text{ AND } \text{NOT } (Q(x)))$$

$$(\exists x)(P(x) \text{ AND } Q(x)) \equiv \text{NOT } (\forall x)(\text{NOT } (P(x)) \text{ OR } \text{NOT } (Q(x)))$$

Observe também que o seguinte é TRUE, onde o símbolo $\Rightarrow$ significa *implica*:

$$(\forall x)(P(x)) \Rightarrow (\exists x)(P(x))$$

$$\text{NOT } (\exists x)(P(x)) \Rightarrow \text{NOT } (\forall x)(P(x))$$

6.6.7 Usando o quantificador universal nas consultas

Sempre que usamos um quantificador universal, é prudente seguir algumas regras para garantir que nossa expressão faça sentido. Discutimos essas regras com relação à consulta C3.

Consulta 3. Listar os nomes dos funcionários que trabalham em *todos* os projetos controlados pelo departamento número 5. Um modo de especificar essa consulta é usar o quantificador universal, conforme mostrado:

C3: $\{f.Unome, f.Pnome \mid$ FUNCIONARIO(f) AND
 $(\forall p)(\text{NOT } (\text{PROJETO}(p)) \text{ OR } \text{NOT } (x.Dnum=5))$
 OR $(\exists t)(\text{TRABALHA_EM}(t) \text{ AND } t.Fcpl=f.Cpf$
 AND $p.Projnumero=t.Pnr)\}$

Podemos desmembrar C3 em seus componentes básicos como se segue:

C3: $\{f.Unome, f.Pnome \mid$ FUNCIONARIO(f) AND F'
 $F' = ((\forall p)(\text{NOT } (\text{PROJETO}(p)) \text{ OR } F_1))$
 $F_1 = \text{NOT } (p.Dnum=5) \text{ OR } F_2$
 $F_2 = ((\exists t)(\text{TRABALHA_EM}(t) \text{ AND } t.Fcpl=f.Cpf$
 AND $p.Projnumero=t.Pnr)\}$

Queremos garantir que um funcionário selecionado f trabalha em *todos* os projetos controlados pelo departamento 5, mas a definição de quantificador universal diz que, para tornar a fórmula quantificada TRUE, a fórmula interna precisa ser TRUE para todas as tuplas no universo. O truque é excluir da quantificação universal todas as tuplas em que não estamos interessados, tornando a condição TRUE para todas essas tuplas. Isso é necessário porque uma variável de tupla universalmente quantificada, como p em C3, precisa ser avaliada como TRUE para cada tupla possível atribuída a ela, para tornar a fórmula quantificada TRUE.

As primeiras tuplas a excluir (fazendo que sejam avaliadas automaticamente como TRUE) são aquelas que não estão na relação R de interesse. Em C3, o uso da expressão $\text{NOT } (\text{PROJETO}(p))$ na fórmula quantificada universalmente avalia como TRUE todas as tuplas x que não estão na relação PROJETO. Depois, excluimos as tuplas em que não estamos interessados da própria R . Em C3, o uso da expressão $\text{NOT } (p.Dnum=5)$ avalia como TRUE todas as tuplas p que estão na relação PROJETO, mas não são controladas pelo departamento 5. Por fim, especificamos uma condição F_2 que precisa ser mantida sobre todas as tuplas restantes em R . Logo, podemos explicar C3 da seguinte forma:

1. Para a fórmula $F' = (\forall p)(F)$ ser TRUE, precisamos fazer que a fórmula F seja TRUE para todas as tuplas no universo que possam ser atribuídas a p . Porém, em C3 só estamos interessados em F ser TRUE para todas as tuplas da relação PROJETO que são controladas pelo departamento 5. Logo, a fórmula F tem a forma $(\text{NOT } (\text{PROJETO}(p)) \text{ OR } F_1)$. A condição $\text{NOT } (\text{PROJETO}(p))$ OR ... é TRUE para todas as tuplas que não estão na relação PROJETO e tem o efeito de eliminar essas tuplas da consideração no valor verdade de F_1 . Para cada tupla na relação PROJETO, F_1 precisa ser TRUE se F' tiver de ser TRUE.

Exemplos Cálculo Relacional de Domínio (CRD)

6.7 O cálculo relacional de domínio

Existe outro tipo de cálculo relacional chamado cálculo relacional de domínio, ou apenas **cálculo de domínio**. Historicamente, enquanto a SQL (ver capítulos 4 e 5), que foi baseada no cálculo relacional de tupla, estava sendo desenvolvida pela IBM Research em San Jose, Califórnia, outra linguagem, chamada QBE (Query-By-Example), que está relacionada ao cálculo de domínio, estava sendo desenvolvida quase simultaneamente no IBM T. J. Watson Research Center em Yorktown Heights, Nova York. A especificação formal do cálculo de domínio foi proposta após o desenvolvimento da linguagem e sistema em QBE.

O cálculo de domínio difere do cálculo de tupla no tipo das variáveis usadas nas fórmulas: em vez de ter variáveis percorrendo as tuplas, elas o fazem por valores isolados dos domínios de atributos. Para formar uma relação de grau n para um resultado de consulta, precisamos ter n dessas variáveis de domínio — uma para cada atributo. Uma expressão do cálculo de domínio tem a forma

$$\{x_1, x_2, \dots, x_n \mid \text{COND}(x_1, x_2, \dots, x_n, x_{n+1}, x_{n+2}, \dots, x_{n+m})\}$$

onde $x_1, x_2, \dots, x_n, x_{n+1}, x_{n+2}, \dots, x_{n+m}$ são variáveis de domínio que percorrem os domínios (dos atributos) e COND é uma condição ou fórmula do cálculo relacional do domínio.

Uma fórmula é composta de átomos. Os átomos de uma fórmula são ligeiramente diferentes daqueles para o cálculo de tupla e podem ser um dos seguintes:

1. Um átomo da forma $R(x_1, x_2, \dots, x_j)$, onde R é o nome de uma relação de grau j e cada x_i , $1 \leq i \leq j$, é uma variável de domínio. Esse átomo declara que uma lista de valores de $\langle x_1, x_2, \dots, x_j \rangle$ deve ser uma tupla na relação cujo nome é R , onde x_i é o valor do i -ésimo valor de atributo da tupla. Para tornar uma expressão de cálculo de domínio mais concisa, podemos *remover as vírgulas* em uma lista de variáveis, escrevendo assim:

$$\{x_1, x_2, \dots, x_n \mid R(x_1 x_2 x_3) \text{ AND } \dots\}$$

no lugar de

$$\{x_1, x_2, \dots, x_n \mid R(x_1, x_2, x_3) \text{ AND } \dots\}$$

2. Um átomo da forma $x_i \text{ op } x_j$, onde op é um dos operadores de comparação no conjunto $\{=, <, \leq, >, \geq, \neq\}$ e x_i e x_j são variáveis de domínio.

3. Um átomo da forma $x_i \text{ op } c$ ou $c \text{ op } x_j$, onde op é um dos operadores de comparação no conjunto $\{=, <, \leq, >, \geq, \neq\}$, x_i e x_j são variáveis de domínio, e c é um valor constante.

Assim como no cálculo de tupla, os átomos são avaliados como TRUE ou FALSE para um conjunto de valores específico, chamados **valores verdade** dos átomos. No caso 1, se as variáveis de domínio receberem valores correspondentes a uma tupla da relação especificada R , então o átomo é TRUE. Nos casos 2 e 3, se as variáveis de domínio receberem valores que satisfazem a condição, então o átomo é TRUE.

De um modo semelhante ao cálculo relacional de tupla, as fórmulas são compostas de átomos, variáveis e quantificadores, de modo que não repetiremos as especificações de fórmulas aqui. A seguir, apresentamos alguns exemplos de consultas especificadas no cálculo de domínio. Usaremos as letras minúsculas $l, m, n, \dots, x, y, z$ para as variáveis de domínio.

Consulta 0. Listar a data de nascimento e o endereço do funcionário cujo nome é 'João B. Silva'.

C0: $\{u, v \mid (\exists q) (\exists r) (\exists s) (\exists t) (\exists w) (\exists x) (\exists y) (\exists z)$
 $(\text{FUNCIONARIO}(qrstuvwxyz) \text{ AND } q = \text{'João'}$
 $\text{AND } r = \text{'B'} \text{ AND } s = \text{'Silva'})\}$

Precisamos de dez variáveis para a relação FUNCIONARIO, uma para percorrer cada um dos domínios dos atributos de FUNCIONARIO em ordem. Das dez variáveis $q, r, s, \dots, z$, somente u e v são livres, pois aparecem à esquerda da barra e, portanto, não devem estar ligadas a um quantificador. Primeiro, especificamos os atributos solicitados, Datanasc e Endereco, pelas variáveis de domínio livres u para DATANASC e v para ENDERECO. Depois, especificamos a condição para selecionar uma tupla após a barra ($\mid$) — a saber, que a sequência de valores atribuídos às variáveis $qrstuvwxyz$ seja uma tupla da relação FUNCIONARIO e que os valores para q (Pnome), r (Micial) e s (Unome) sejam iguais a 'João', 'B' e 'Silva', respectivamente. Por conveniência, quantificaremos apenas as variáveis que *realmente aparecem em uma condição* (estas seriam q, r e s em C0) no restante de nossos exemplos.¹⁴

Uma notação abreviada alternativa, usada em QBE, para escrever essa consulta, é atribuir as constantes 'João', 'B' e 'Silva' diretamente como mostramos em C0A. Aqui, todas as variáveis que não aparecem à esquerda da barra são implicitamente quantificadas de maneira existencial.¹⁵

¹⁴ Observe que a notação de quantificar apenas as variáveis de domínio realmente usadas nas condições e de mostrar um predicado como FUNCIONARIO(qrstuvwxyz) sem separar as variáveis de domínio com vírgulas é uma notação abreviada, utilizada por conveniência; essa não é a notação formal correta.

¹⁵ Mais uma vez, esta não é uma notação formalmente precisa.

C0A: $\{u, v \mid \text{FUNCIONARIO}('João', 'B', 'Silva', t, u, v, w, x, y, z)\}$

Consulta 1. Recuperar o nome e o endereço de todos os funcionários que trabalham para o departamento 'Pesquisa'.

C1: $\{q, s, v \mid (\exists z)(\exists l)(\exists m)(\text{FUNCIONARIO}(qrstuvwxyz) \text{ AND DEPARTAMENTO}(lmno) \text{ AND } l = 'Pesquisa' \text{ AND } m = z)\}$

Uma condição relacionando duas variáveis de domínio que percorra os atributos de duas relações, como $m = z$ em C1, é uma **condição de junção**, enquanto uma condição que relaciona uma variável de domínio a uma constante, como $l = 'Pesquisa'$, é uma **condição de seleção**.

Consulta 2. Para cada projeto localizado em 'Mauá', listar o número do projeto, o número do departamento de controle e último nome, data de nascimento e endereço do gerente do departamento.

C2: $\{i, k, s, u, v \mid (\exists j)(\exists m)(\exists n)(\exists t)(\text{PROJETO}(hijk) \text{ AND FUNCIONARIO}(qrstuvwxyz) \text{ AND DEPARTAMENTO}(lmno) \text{ AND } k = m \text{ AND } n = t \text{ AND } j = 'Mauá')\}$

Consulta 6. Listar os nomes dos funcionários que não têm dependentes.

C6: $\{q, s \mid (\exists t)(\text{FUNCIONARIO}(qrstuvwxyz) \text{ AND } (\text{NOT}(\exists l)(\text{DEPENDENTE}(lmnop) \text{ AND } t = l)))\}$

C6 pode ser reformulada usando quantificadores universais no lugar dos quantificadores existenciais, como mostramos em C6A:

C6A: $\{q, s \mid (\exists t)(\text{FUNCIONARIO}(qrstuvwxyz) \text{ AND } ((\forall l)(\text{NOT}(\text{DEPENDENTE}(lmnop) \text{ AND } t = l))) \text{ OR } \text{NOT}(t = l)))\}$

Consulta 7. Listar os nomes dos gerentes que têm pelo menos um dependente.

C7: $\{s, q \mid (\exists t)(\exists j)(\exists l)(\text{FUNCIONARIO}(qrstuvwxyz) \text{ AND DEPARTAMENTO}(hijk) \text{ AND DEPENDENTE}(lmnop) \text{ AND } t = j \text{ AND } l = t)\}$

Como dissemos anteriormente, pode-se mostrar que qualquer consulta que pode ser expressa na álgebra relacional básica também pode ser expressa no cálculo relacional de domínio ou de tupla. Além disso, qualquer *expressão segura* no cálculo relacional de domínio ou de tupla pode ser expressa na álgebra relacional básica.

A linguagem QBE foi baseada no cálculo relacional de domínio, embora isso fosse observado mais tarde, depois que o cálculo de domínio foi formalizado. A QBE foi uma das primeiras linguagens de consulta gráficas com sintaxe mínima desenvolvidas para sistemas de banco de dados. Ela foi desenvolvida na IBM Research e está disponível como um produto comercial da empresa como parte da opção de interface Query Management Facility (QMF) para DB2. As ideias básicas usadas na QBE têm sido aplicadas em vários outros produtos comerciais. Devido a seu lugar importante na história das linguagens relacionais, incluímos uma visão geral da QBE no Apêndice C.

Conclusão

O cálculo relacional é um componente essencial da teoria dos bancos de dados, fornecendo a base lógica para consultas em sistemas relacionais. Sua abordagem declarativa permite maior abstração, eficiência e independência em relação ao método de execução, tornando o processo de manipulação de dados mais simples e flexível. Além disso, seu estudo é fundamental para profissionais da área de tecnologia, pois contribui significativamente para a compreensão da linguagem SQL, da otimização de consultas e da modelagem adequada de dados. Esses conhecimentos são indispensáveis para o desenvolvimento de sistemas eficientes, seguros e capazes de lidar com grandes volumes de informação. Outro aspecto relevante é que o cálculo relacional está diretamente ligado à garantia da integridade e consistência dos dados, fatores essenciais para a confiabilidade das informações em qualquer sistema. Sua base matemática proporciona maior rigor no tratamento dos dados, reduzindo erros e inconsistências. Por fim, mesmo não sendo utilizado de forma explícita no cotidiano dos desenvolvedores, seus conceitos estão presentes em praticamente todos os sistemas modernos que utilizam bancos de dados relacionais. Dessa forma, compreender o cálculo relacional não apenas fortalece a base teórica do profissional, mas também melhora sua capacidade de desenvolver soluções mais eficientes, organizadas e alinhadas às boas práticas da área de tecnologia.

Referências Bibliográficas

IME-USP. **Cálculo Relacional**. São Paulo: Instituto de Matemática e Estatística da Universidade de São Paulo, s.d. Disponível em: <<https://www.ime.usp.br/~jef/calcrelac.pdf>>. Acesso em: 27 mar. 2026.

IBM. **O que é um banco de dados relacional?** s.d. Disponível em: <https://www.ibm.com/br-pt/topics/relational-databases>. Acesso em: 27 mar. 2026.

IBM. **Edgar F. Codd**. s.d. Disponível em: <https://www.ibm.com/history/edgar-codd>. Acesso em: 27 mar. 2026.

COMPUTER WEEKLY. **Banco de dados relacional**. Disponível em: . Acesso em: 27 mar. 2026.

PROFESSOR LEOMIR. **Álgebra e cálculo relacional**. s.d. Disponível em: <https://professorleomir.wordpress.com/wp-content/uploads/2011/08/algebra-e-caculo-relacional-aula-5.pdf>. Acesso em: 27 mar. 2026.